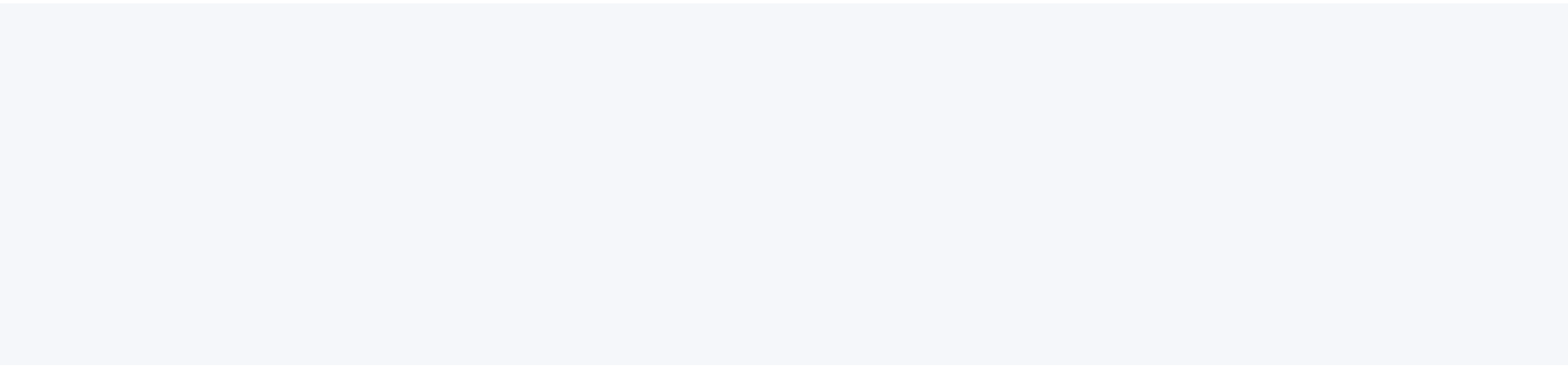


Синхронизация данных (State Sync)

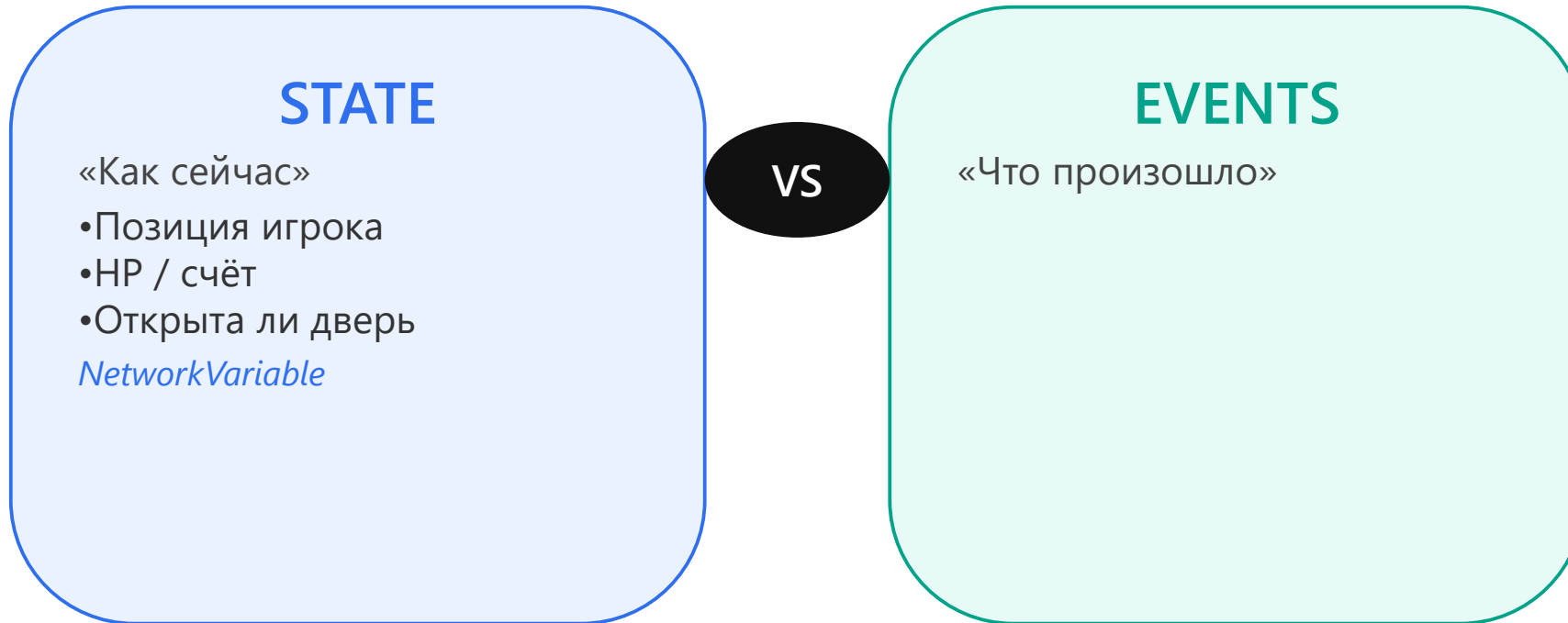
Unity Netcode for GameObjects (NGO)

Лекция 2: NetworkVariable, сериализация, оптимизация трафика

Евгений Малько · МИРЭА · 2026



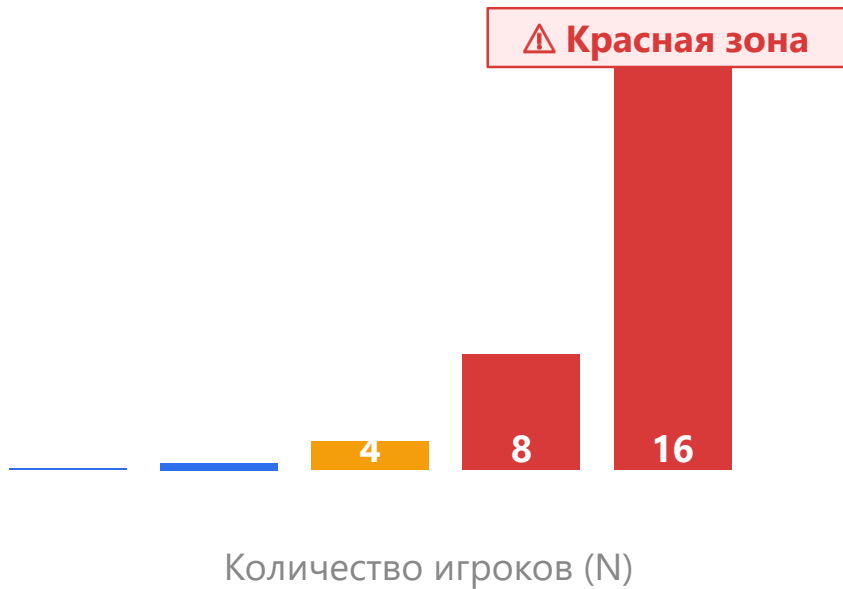
Что такое State Sync?



💡 Назовите 3 состояния вашей игры, которые должны совпадать у всех

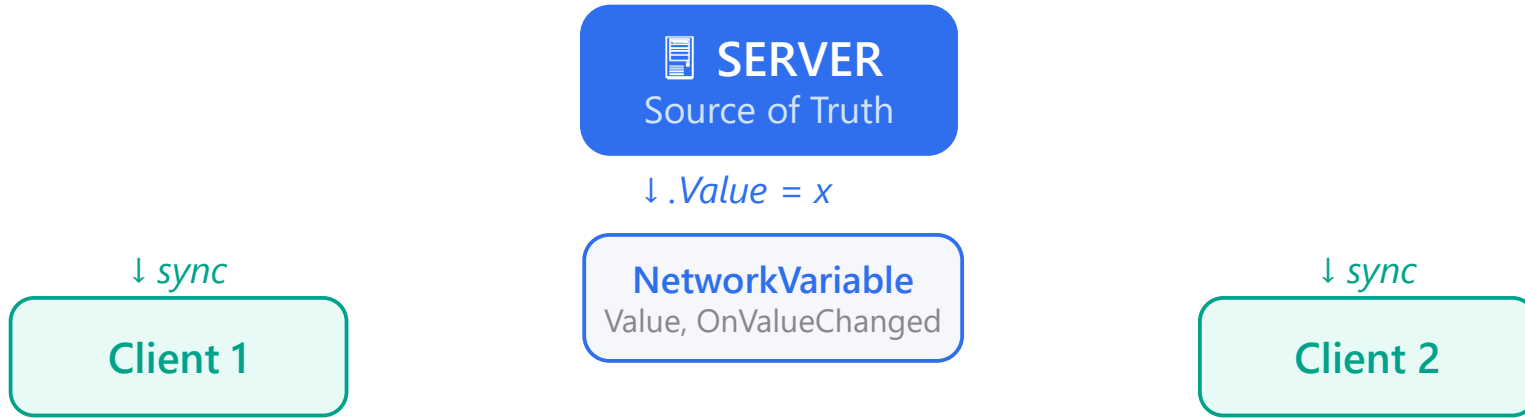
Почему «слать каждый кадр» — плохо

Рост трафика при N клиентах



- Сеть $\neq$ рендер: частота сети должна быть ниже
- Каждый лишний апдейт = трафик + лаг + потери
- Нужно: пороги + частота + выбор получателей

NetworkVariable: идея и жизненный цикл



✓ Late-join клиент получает текущее значение автоматически • Подписка `OnValueChanged` — в `OnNetworkSpawn`

OnValueChanged: правильный паттерн



```
public override void OnNetworkSpawn() {  
    score.OnValueChanged += OnScore;  
    OnScore(0, score.Value); // initial UI update  
}  
public override void OnNetworkDespawn() {  
    score.OnValueChanged -= OnScore;  
}
```

✓Подписываемся в OnNetworkSpawn • Отписываемся в OnNetworkDespawn • UI обновляем сразу текущим Value

Read/Write Permissions — почему «не обновляется»

Read:	✓ Чаше	✓ Редко (Owner-
Read: Owner	✗ Клиент не видит	✓ Личные данные
Read: Server	✗ Лишнее	✗ Не имеет смысла

Типичные кейсы

→Server-authoritative: WritePerm = Server

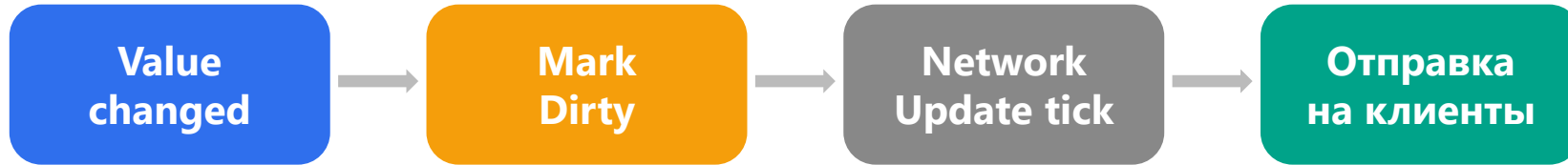
→Owner-authoritative (редко): WritePerm = Owner

→Read: Everyone — для UI и реплея

⚠ Неверный WritePerm = читерство

```
new NetworkVariable<int>(0, NetworkVariableReadPermission.Everyone,  
NetworkVariableWritePermission.Server);
```

Когда реально уходит по сети: Dirty/Send



⚠ Значение **не изменилось** — dirty не устанавливается — **ничего не отправляется**

Оптимизация #1

Не трогать .Value без необходимости

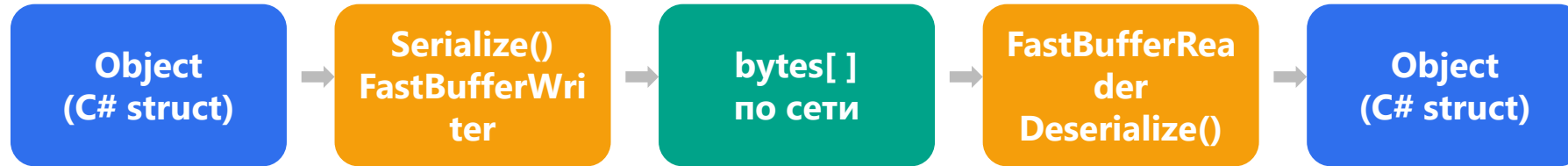
Оптимизация #2

Менять .Value реже (FixedUpdate, не Update)

Дисциплина кода

Не присваивать одно значение каждый кадр

Сериализация: объект → байты → объект



- Сериализация = упаковка данных в байты для передачи по сети
- Десериализация = распаковка на стороне получателя
- Порядок write/read должен совпадать — иначе мусор в буфере!

⚠ Write(A, B, C) на сервере vs Read(A, B) на клиенте = undefined behavior

FastBufferWriter / Reader (низкий уровень)

 **Высокая скорость**

 **Типобезопасность**

Все типы известны на этапе компиляции

 **Контроль формата**

Вы решаете порядок и размер каждого поля

FastBufferWriter



buffer[]



FastBufferReader

INetworkSerializable: кастомные типы

```
public struct PlayerState :  
    INetworkSerializable {  
    public Vector3 Position;  
    public Quaternion Rotation;  
    public byte Stance;  
    public void NetworkSerialize<T>(  
        BufferSerializer<T> s)  
        where T : IReaderWriter {  
        s.SerializeValue(ref Position);  
        s.SerializeValue(ref Rotation);  
        s.SerializeValue(ref Stance);  
    }  
}
```

Когда использовать

- ✓ Struct состояния игрока
- ✓ Данные предмета/инвентаря
- ✓ Работает в NV и RPC

*Порядок полей
обязателен!*

Custom Serialization: когда нужно



Меньше байт

Квантизация: позиция в см (short), угол (ushort). Экономия до 50%



Быстрее

Packed числа вместо int32: 1–3 байта вместо 4 для малых значений



Контроль формата

Собственный бинарный формат для дорогих и частых данных

Данных много? **Упакуйте компактнее** — выигрыш будет кратным.
Всегда **измеряйте до/после**

Классическая ошибка: чтение не того формата

SERVER: Write()

```
writer.WriteValue(A);  
writer.WriteValue(B);  
writer.WriteValue(C);
```



CLIENT: Read() ✗

```
reader.ReadValue(ref A);  
reader.ReadValue(ref B);  
С забыли — буфер смещён!
```

- Порядок и размер полей должны совпадать на сервере и клиенте
- Переменная длина (string/array) требует особой аккуратности
- Всегда тестируйте формат на 2 клиентах**

3 рычага оптимизации трафика



FREQUENCY

*Как часто
отправляем*

Пороги ϵ ,
ограничение частоты,
FixedUpdate вместо
Update.

Самый быстрый
выигрыш —
начинайте здесь.

★ **Начинайте здесь!**



RELEVANCY

Кому отправляем

Отправляем только
видящим клиентам.
AOI (Area of Interest)
для большого мира.



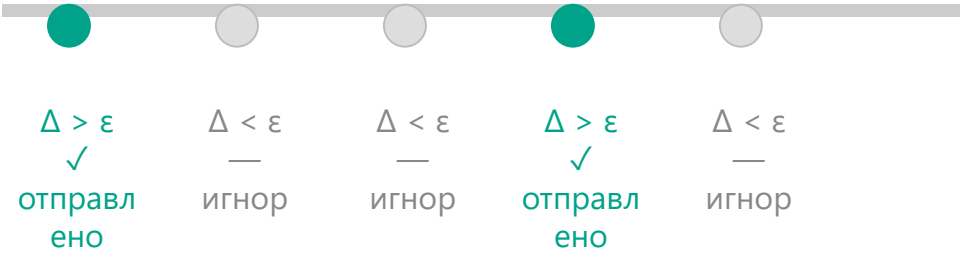
PAYLOAD

Сколько байт

Квантизация, packed
int,
INetworkSerializable,
FastBufferWriter.

Frequency: пороги и «send only when changed»

Dead zone ε (порог отправки)



```
// threshold  
const float E = 0.02f;  
if ((pos-last).sqrMagnitude  
    < E*E) return;  
netPos.Value = pos;  
last = pos;
```

✓ Порог в 2 см незаметен визуально, но **кратно экономит трафик**

Payload: квантизация (float → int)

Позиция X float32 = 4 байта → short (см) = 2 байта -50%

Угол Yaw float32 = 4 байта → ushort (0-65535) = 2 байта -50%

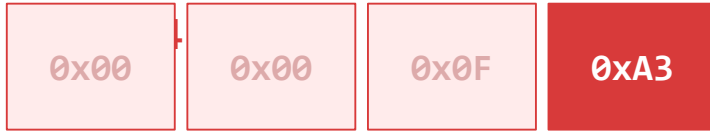
Позиция 3D 3× float = 12 байт → 3× short = 6 байт -50%

Идея квантизации

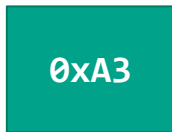
Игрок не замечает разницу в 1–2 см. Сеть «видит» экономию в байтах.

Сознательно снижаем точность ради стабильности сети.

Packed числа: экономим байты на int



packed int = 1–3 байта (varint)



Где применять packed int

- ID сущностей (обычно малые числа)
- Счёт, очки, количество патронов
- Битовые флаги состояний

*Маленькая экономия × 100
объектов =
большой выигрыш*

NetworkTransform: почему может «жрать трафик»

NetworkTransform

Unreliable Deltas	✓ ON
Quaternion Compression	X OFF
Half Float Precision	X OFF
Sync Position X/Y/Z	✓ ✓ ✓
Sync Rotation X/Y/Z	✓ ✓ ✓
Network Tick Rate	60

Что влияет на трафик

→ Unreliable Deltas → только изменения

→ Quaternion Compression → меньше байт на ротацию

→ Half Float → 2 байта вместо 4

⚠ Некоторые изменения триггерят full-state update!

Итог: почему нельзя слать каждый кадр

60Hz без порогов/ограничений	Пороги ϵ + ограничение частоты
float везде, всем и всегда	Компактные типы + квантизация
State для разовых событий	События — через RPC
NV без OnValueChanged в UI	OnValueChanged для UI

Практика: NV для HP (server-authoritative)

```
// Read: Everyone (all clients see HP), Write: Server (only server sets HP)
public NetworkVariable<int> HP = new(100, Everyone, Server);
[ServerRpc(RequireOwnership = false)]
void DealDamageServerRpc(int dmg) {
    HP.Value = Mathf.Max(0, HP.Value - dmg);
}
override void OnNetworkSpawn() {
    HP.OnValueChanged += OnHP; OnHP(0, HP.Value);
}
```

Сервер решает · клиенты отображают · UI слушает OnValueChanged

Практика: сложное состояние PlayerState

Vector3 Position → 6 байт (*short*×3)

Quaternion Rotation → 8 байт
(*ushort*×4)

byte Stance → 1 байт

Итого: ~15 байт vs **raw** 40 байт для 3
float-тройок

Архитектура

✓struct + INetworkSerializable
✓NV<PlayerState> пишет
сервер

✓Клиент читает +
интерполирует визуал

*Синхронизировать целым
пакетом,
но реже и только при
изменениях*

События vs State: где RPC, где NV

⚡ Событие
Shoot = RPC

💾 STATE
Ammo = NV

Выстрел — разовое событие. Патроны — всегда актуальны.

⚡ Событие
Pickup = RPC

💾 STATE
InventoryState = NV

Подбор — произошёл один раз. Инвентарь — текущее состояние.

⚡ Событие
PlaySound = RPC

💾 STATE
IsAlive = NV

Звук не нужно воспроизводить при коннекте. Статус — нужен.

⚠ Перепутать — значит получить лишний трафик или пропуск событий при позднем подключении

Чек-лист оптимизации (7 пунктов)

1. ☐ Не меняю .Value без нужды

2. ☐ Есть ϵ -порог перед присвоением

3. ☐ Частота апдейтов ограничена (FixedUpdate / таймер)

4. ☐ Компактные типы: byte/short/packed вместо int32

5. ☐ Сложные состояния: INetworkSerializable

6. ☐ UI слушает только OnValueChanged

7. ☐ Разовые события — через RPC, не NV

 Сделайте первые 3 пункта — уже будет значительно лучше

Что мы узнали

